

SISTEMA ESPECIALISTA EM GRC

RAG + LLM com Base no COSO ERM 2017

Documentação Técnica de Alto Nível

Fevereiro de 2026

Sumário

1. Visão Geral do Sistema	4
1.1 Objetivo Principal	4
1.2 Visão Arquitetural	4
2. Base de Conhecimento — COSO ERM 2017	5
2.1 Pipeline de Ingestão de Documentos	5
2.1.1 Carregamento de Arquivos	5
2.1.2 Fragmentação de Textos (Chunking)	5
2.1.3 Deduplicação por Hash (Hash Dedup)	5
2.1.4 Deduplicação Semântica (Semantic Dedup)	5
2.1.5 Resultados da Ingestão	6
3. Modelos de Embeddings e Indexação Vetorial	7
3.1 Modelo de Embeddings — BAAI/bge-m3	7
3.2 Banco de Dados Vetorial — ChromaDB	7
3.3 Estratégia de Recuperação — Max Marginal Relevance (MMR)	7
4. Re-Ranking Semântico	9
4.1 Modelo de Re-Ranking — BAAI/bge-reranker-v2-m3	9
4.2 Por que usar Re-Ranking?	9
5. Sistema de Memória Semântica Persistente	10
5.1 Visão Geral	10
5.2 Modelo de Embeddings da Memória — sentence-transformers/all-MiniLM-L6-v2	10
5.3 Parâmetros e Ciclo de Vida da Memória	10
5.4 Fluxo de Gestão de Memória	10
6. Modelo de Linguagem — Meta Llama 3.2 3B Instruct	12
6.1 Especificações do Modelo	12
6.2 Técnica de Quantização — BitsAndBytes NF4	12
6.3 Parâmetros de Geração	12
6.4 Estrutura do Prompt — Llama 3.2 Chat Template	13
7. Métricas de Avaliação	14
7.1 Avaliação de Qualidade da Resposta — Perplexidade	14
7.1.1 Metodologia	14
7.1.2 Interpretação das Métricas	14
7.2 Avaliação da Pipeline RAG — RAGEvaluator	14
7.2.1 Modelo de Embeddings para Avaliação	15
7.2.2 Métricas Implementadas	15
7.2.3 Verificação de Relevância Semântica	15

7.2.4 Análise por Threshold	15
8. Fluxo Completo de uma Interação	17
9. Stack Tecnológico Completo	18
10. Considerações Técnicas e Decisões de Design	19
10.1 Rastreabilidade Total das Respostas	19
10.2 Uso de Temperatura Baixa (0.3)	19
10.3 Isolamento Completo — Sistema On-Premises	19
10.4 Pipeline de Dois Estágios (MMR + Re-Rank)	19
10.5 Gestão Automática de Memória	19

1. Visão Geral do Sistema

Este documento descreve em detalhe a arquitetura, os componentes técnicos, as técnicas de recuperação de informação, os modelos de linguagem e as métricas de avaliação do sistema especialista em Governança, Risco e Compliance (GRC), com foco específico no framework COSO ERM 2017 (Committee of Sponsoring Organizations of the Treadway Commission — Enterprise Risk Management).

O sistema combina um Large Language Model (LLM) local baseado na família Meta Llama com uma pipeline de Retrieval-Augmented Generation (RAG), memória semântica persistente e mecanismos de avaliação de qualidade das respostas. A arquitetura foi projetada para garantir respostas altamente fundamentadas na base de conhecimento do COSO ERM 2017, minimizando alucinações e maximizando a rastreabilidade das fontes citadas.

1.1 Objetivo Principal

Fornecer respostas especializadas e auditáveis sobre o COSO ERM 2017, utilizando exclusivamente o conteúdo do framework como base de conhecimento. O sistema deve ser capaz de:

- Responder perguntas técnicas sobre Gestão de Riscos Corporativos
- Persistir o histórico de interações para enriquecer o contexto de inferência do modelo em interações futuras.
- Avaliar a qualidade e confiança das respostas geradas automaticamente
- Operar inteiramente em ambiente local (on-premises), sem dependência de APIs externas

1.2 Visão Arquitetural

A arquitetura do sistema é composta por cinco camadas funcionais independentes que colaboram na geração de respostas:

Camada 1	Armazenamento Vetorial (ChromaDB + BAAI/bge-m3)
Camada 2	Re-Ranking Semântico (BAAI/bge-reranker-v2-m3)
Camada 3	Memória de Conversação Persistente (ChromaDB + multilingual-e5-small)
Camada 4	Geração de Linguagem (Llama 3.2 3B Instruct — Quantizado 4-bit)
Camada 5	Avaliação de Qualidade (Perplexidade + Métricas RAG)

2. Base de Conhecimento — COSO ERM 2017

A base de conhecimento do sistema é construída integralmente a partir do documento oficial do COSO ERM 2017 (Enterprise Risk Management — Integrating with Strategy and Performance). O texto é carregado, fragmentado (chunked), deduplicado e indexado em um banco de dados vetorial que serve como a única fonte de verdade do sistema.

2.1 Pipeline de Ingestão de Documentos

O processo de ingestão é implementado na classe PDFVectorStore (BuildVectorStore.py) e segue as seguintes etapas:

2.1.1 Carregamento de Arquivos

O sistema suporta arquivos no formato `.txt`, utilizando o `TextLoader` da biblioteca LangChain. Durante o processo de ingestão, cada documento recebe um metadado de origem (`source`), que é propagado para os chunks gerados na etapa de segmentação. Dessa forma, mesmo com a inclusão futura de novos documentos na vector store, é possível rastrear cada chunk até seu documento de origem.

2.1.2 Fragmentação de Textos (Chunking)

Os documentos são divididos em fragmentos menores utilizando o `RecursiveCharacterTextSplitter` com os seguintes parâmetros:

chunk_size	400 tokens por fragmento
chunk_overlap	80 tokens de sobreposição entre fragmentos
Estratégia	Recursiva: tenta quebrar em parágrafos, depois em sentenças, depois em palavras

A sobreposição de 80 tokens (overlap) garante que informações que apareçam no limite entre dois chunks sejam capturadas em ambos os fragmentos, evitando perda de contexto semântico em transições de parágrafos.

2.1.3 Deduplicação por Hash (Hash Dedup)

Após o chunking, todos os fragmentos passam por uma deduplicação baseada em hash SHA-256. O conteúdo de cada chunk é normalizado para letras minúsculas antes do cálculo do hash, garantindo a eliminação de cópias idênticas (independente de capitalização). No documento COSO ERM 2017 processado, foram identificados e removidos 0 chunks duplicados por hash.

2.1.4 Deduplicação Semântica (Semantic Dedup)

Após a deduplicação por hash, uma segunda etapa de deduplicação semântica é executada. Todos os chunks são convertidos em embeddings e a similaridade cosseno entre cada par é calculada. Chunks com similaridade superior a 0.95 são considerados semanticamente equivalentes e o duplicado é removido. Nesta etapa, foram removidos adicionalmente 8 chunks semanticamente redundantes do documento COSO ERM 2017.

2.1.5 Resultados da Ingestão

Ao final do pipeline de ingestão, o documento COSO ERM 2017 resultou nas seguintes estatísticas:

Etapa	Quantidade
Documentos carregados	1
Chunks gerados (pós-split)	301
Removidos por hash dedup	0
Removidos por semantic dedup	8
Chunks finais indexados	293

3. Modelos de Embeddings e Indexação Vetorial

3.1 Modelo de Embeddings — BAAI/bge-m3

O modelo de embeddings utilizado para indexação do conhecimento e para recuperação de contexto RAG é o BAAI/bge-m3, desenvolvido pelo Beijing Academy of Artificial Intelligence (BAAI). Este modelo foi selecionado por suas características superiores em tarefas de recuperação de informação multilíngue:

Modelo	BAAI/bge-m3
Dimensão dos embeddings	1.024 dimensões
Suporte multilíngue	Mais de 100 idiomas, incluindo Português
Comprimento máximo	8.192 tokens por documento
Dispositivo	GPU (CUDA) para inferência acelerada
Uso no sistema	Indexação de chunks + similaridade na busca RAG + deduplicação semântica

3.2 Banco de Dados Vetorial — ChromaDB

O armazenamento e a recuperação dos vetores de embedding são gerenciados pelo ChromaDB, um banco de dados vetorial open-source otimizado para aplicações de IA. O banco é persistido em disco no diretório llama/RAG/db, permitindo que o índice seja carregado entre execuções sem necessidade de reprocessar os documentos.

3.3 Estratégia de Recuperação — Max Marginal Relevance (MMR)

A recuperação dos chunks mais relevantes não utiliza apenas similaridade cosseno direta. O sistema emprega a estratégia Max Marginal Relevance (MMR), que equilibra dois objetivos:

- Relevância: o documento deve ser semanticamente próximo à query do usuário
- Diversidade: os documentos recuperados devem ser distintos entre si, evitando redundância

Os parâmetros configurados para a busca MMR são:

k (documentos retornados)	20 chunks para análise pelo Re-ranker
fetch_k (pré-seleção interna)	50 chunks analisados internamente pelo ChromaDB
lambda_mult	0.5 (balanceamento igual entre relevância e diversidade)

4. Re-Ranking Semântico

4.1 Modelo de Re-Ranking — BAAI/bge-reranker-v2-m3

Após a recuperação inicial dos 20 chunks pelo ChromaDB, o sistema aplica um segundo estágio de ordenação utilizando um modelo de Re-Ranking especializado. O re-ranker utiliza o modelo BAAI/bge-reranker-v2-m3, um modelo de cross-encoder treinado especificamente para a tarefa de relevância par query-documento.

Modelo	BAAI/bge-reranker-v2-m3
Arquitetura	Cross-Encoder (analisa query e documento juntos)
Dispositivo	GPU (CUDA)
Input	Pares [query, chunk_text]
Output	Score de relevância (logit) por par
top_n selecionado	6 documentos mais relevantes

4.2 Por que usar Re-Ranking?

Modelos de embedding (bi-encoders como o bge-m3) codificam query e documento de forma independente, o que é eficiente mas pode introduzir imprecisões semânticas. O cross-encoder do re-ranker analisa a query e o documento juntos em uma única inferência, capturando interações mais sutis entre os termos. O fluxo completo é:

Etapa	Modelo	Input	Output
1ª Recuperação (Recall)	bge-m3 (bi-encoder)	Query → embedding	Top 20 chunks por similaridade MMR
2ª Ordenação (Precision)	bge-reranker-v2-m3 (cross-encoder)	Pares [query, chunk]	Top 6 chunks reordenados por relevância real

Essa abordagem em duas etapas é chamada de two-stage retrieval e é uma das técnicas de maior impacto na qualidade de sistemas RAG. A primeira etapa prioriza recall (não perder documentos relevantes), enquanto a segunda prioriza precision (garantir que os documentos entregues ao LLM sejam os mais relevantes).

5. Sistema de Memória Semântica Persistente

5.1 Visão Geral

O sistema incorpora um mecanismo de memória de conversação persistente implementado via ChromaDB, que armazena interações anteriores e as recupera de forma semântica para enriquecer o contexto das respostas. A memória é gerenciada por três componentes principais:

Memory_support	Gerenciador principal: armazena, consulta e monta o bloco de memória para o prompt
MemoryCleaner	Remove memórias duplicadas ou muito curtas via deduplicação semântica
MemoryPurifier	Força o limite máximo de memórias, removendo as mais antigas quando necessário

5.2 Modelo de Embeddings da Memória — `intfloat/multilingual-e5-small`

Para o sistema de memória, um modelo de embedding mais leve é utilizado, o `intfloat/multilingual-e5-small`, que oferece boa qualidade semântica com menor custo computacional, adequado para operações frequentes de escrita e leitura na memória.

Modelo	<code>intfloat/multilingual-e5-small</code>
Dimensão	384 dimensões
Uso	Embeddings de perguntas e respostas para memória semântica
Dispositivo	GPU (CUDA)

5.3 Parâmetros e Ciclo de Vida da Memória

threshold_distance	0.70 — distância máxima para considerar uma memória relevante para a query atual
max_memories	300 — limite máximo de memórias armazenadas
similarity_threshold (cleaner)	0.95 — similaridade mínima para considerar duas memórias como duplicadas
min_length (cleaner)	15 caracteres — tamanho mínimo para uma memória ser válida

5.4 Fluxo de Gestão de Memória

A cada nova interação, o seguinte ciclo de gestão de memória é executado:

- Nova memória (pergunta + resposta do LLM) é gerada com embedding e salva no ChromaDB
- Deduplicação semântica: memórias com similaridade ≥ 0.80 são removidas
- Purificação: se o total ultrapassar 300 memórias, as mais antigas são descartadas
- Na próxima consulta: as K=3 memórias mais próximas semanticamente da nova query são recuperadas e injetadas no prompt
- Ao utilizar somente 3 memórias é possível deixar o prompt mais enxuto e com o foco no contexto do RAG que é o mais importante, garantindo que o modelo não fique perdido “Lost in The Middle”.

6. Modelo de Linguagem — Meta Llama 3.2 3B Instruct

6.1 Especificações do Modelo

O modelo de linguagem utilizado para geração de respostas é o Meta Llama 3.2 3B Instruct, um modelo de linguagem autoregressivo de 3 bilhões de parâmetros otimizado para seguir instruções. Para viabilizar a execução em hardware com memória GPU limitada, o modelo é quantizado utilizando a técnica NF4 (Normal Float 4-bit) via biblioteca BitsAndBytes.

Modelo base	meta-llama/Llama-3.2-3B-Instruct
Parâmetros	3 bilhões
Quantização	4-bit NF4 (Normal Float 4-bit)
Double Quantization	Habilitada (maior redução de memória)
dtype de computação	bfloat16
Device map	CUDA (GPU completa)
Modo de operação	Inferência apenas (model.eval())
Total VRAM Utilizada	8gb de VRAM

6.2 Técnica de Quantização — BitsAndBytes NF4

A quantização NF4 é uma técnica avançada de compressão de modelos desenvolvida especificamente para modelos de linguagem de grande porte. Diferente da quantização inteira convencional, o NF4 utiliza uma distribuição de pesos otimizada para a distribuição gaussiana típica dos pesos de transformers, minimizando a degradação de qualidade durante a compressão.

Com a quantização 4-bit + double quantization habilitada, o modelo Llama 3.2 3B que originalmente requeria ~6GB de VRAM passa a operar com aproximadamente 2-3GB de VRAM, viabilizando o uso em GPUs consumer de 8GB ou mais.

6.3 Parâmetros de Geração

temperature	0.3 — baixa temperatura para respostas mais determinísticas e fiéis ao contexto
max_new_tokens	512 tokens gerados por resposta
top_p (nucleus sampling)	0.7 — considera tokens até 70% da probabilidade acumulada
top_k	40 — considera apenas os 40 tokens mais prováveis a cada passo
do_sample	True — amostragem estocástica habilitada

6.4 Estrutura do Prompt — Llama 3.2 Chat Template

O prompt enviado ao modelo segue rigorosamente o template de chat do Llama 3.2, com os delimitadores especiais de início/fim de texto e cabeçalhos de turno. A estrutura é composta por três blocos de contexto injetados dinamicamente:

Bloco	Conteúdo	Fonte
CONTEXT_MEMORY	Até 3 memórias de conversações anteriores relevantes	ChromaDB de memória semântica
CONTEXT_RAG	Os 6 chunks mais relevantes do COSO ERM 2017	ChromaDB RAG após re-ranking
user	Pergunta atual do usuário	Input do usuário

As regras obrigatórias injetadas no system prompt forçam o modelo a: usar informações do CONTEXT_RAG, e quando pertinente usar informações do CONTEXT_MEMORY, declarar explicitamente quando uma informação não foi encontrada no contexto — eliminando assim alucinações baseadas no conhecimento paramétrico do modelo.

7. Métricas de Avaliação

O sistema incorpora dois módulos de avaliação complementares: avaliação de qualidade da resposta gerada (Perplexidade) e avaliação da pipeline de recuperação RAG (Hit Rate, MRR e Precision@1).

7.1 Avaliação de Qualidade da Resposta — Perplexidade

A classe `CalculatePerplexity` (`perplexity.py`) avalia a qualidade e coerência de cada resposta gerada pelo LLM em tempo real, utilizando os próprios logits do modelo durante a geração.

7.1.1 Metodologia

A perplexidade é calculada sobre os tokens gerados (excluindo o prompt de entrada) utilizando os log-probabilities do modelo:

- Log-probs são extraídos aplicando `log_softmax` sobre os logits do modelo
- Para cada token gerado, é coletado o log-prob do token efetivamente escolhido
- O Mean Log-Probability é a média dos log-probs de todos os tokens gerados
- A Perplexidade é calculada como $\exp(-\text{mean_logprob})$

7.1.2 Interpretação das Métricas

Métrica	Fórmula	Interpretação
Mean Log-Probability	$\text{mean}(\log P(t_i))$	Confiança média do modelo por token. Valores próximos a 0 indicam alta confiança.
Perplexity (PPL)	$\exp(-\text{mean_logprob})$	Quanto menor, mais coerente e confiante é a resposta. PPL < 10 indica boa fluência.
Pairs (token, logprob)	Lista de pares	Permite identificar tokens específicos onde o modelo teve baixa confiança (possíveis erros).

Os logits já são produzidos durante a geração com `return_dict_in_generate=True`.

7.2 Avaliação da Pipeline RAG — RAGEvaluator

A classe `RAGEvaluator` (`EvaluationRAG.py`) implementa um framework de avaliação offline da qualidade de recuperação da pipeline RAG. A avaliação é conduzida sobre um dataset de pares (query, chunks_relevantes) e mede quanto do conhecimento relevante o sistema consegue recuperar.

7.2.1 Modelo de Embeddings para Avaliação

O julgamento de relevância entre documentos recuperados e documentos esperados é feito semanticamente via o modelo BAAI/bge-m3, garantindo que variações parafrásticas de um mesmo conceito sejam reconhecidas como equivalentes — e não apenas matches exatos de texto.

7.2.2 Métricas Implementadas

Métrica	Definição	Fórmula	Interpretação
Hit Rate	Proporção de queries em que pelo menos 1 documento relevante foi recuperado no Top-K	$\text{hits} / \text{n_queries}$	Mede o recall geral da pipeline. 1.0 = perfeito.
MRR (Mean Reciprocal Rank)	Média do inverso da posição do primeiro documento relevante na lista	$\text{mean}(1 / \text{rank_first_hit})$	Mede quão cedo o primeiro documento relevante aparece. MRR=1.0 sempre no 1º lugar.
Precision@1 (P@1)	Proporção de queries em que o 1º documento retornado é relevante	$\text{p1_hits} / \text{n_queries}$	Mede a precisão do topo da lista. Crucial para sistemas que usam apenas o top resultado.

7.2.3 Verificação de Relevância Semântica

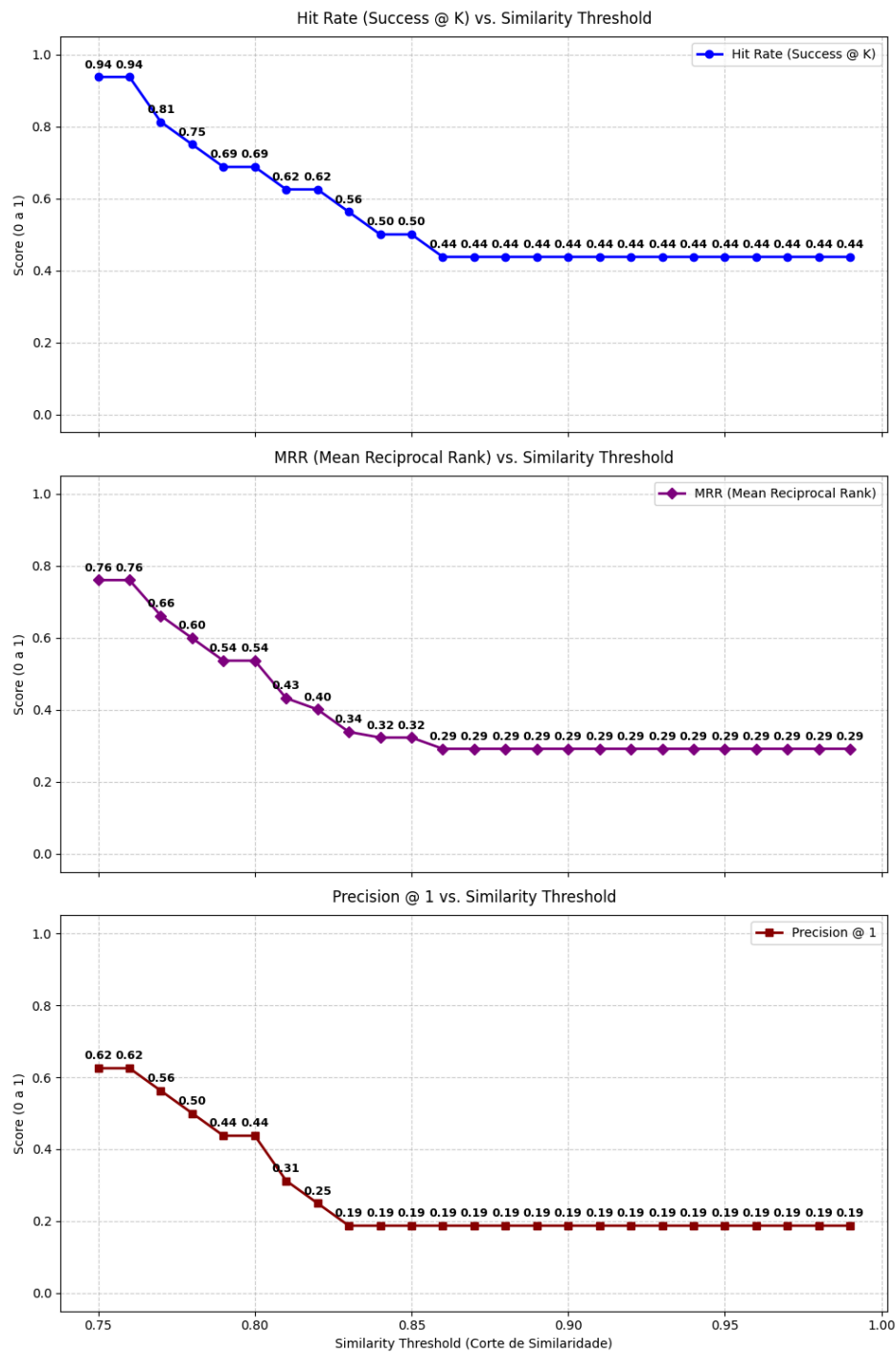
A relevância de um documento recuperado em relação aos documentos esperados (ground truth) é verificada pelo método `_semantic_check`, que:

- Gera embeddings para todos os documentos recuperados e para todos os documentos relevantes esperados
- Calcula a matriz de similaridade cosseno entre os dois conjuntos via produto matricial
- Para cada documento relevante esperado, identifica o documento recuperado de maior similaridade
- Considera o documento relevante como 'encontrado' se a similaridade máxima for $\geq \text{similarity_threshold}$

7.2.4 Análise por Threshold

O avaliador executa uma varredura de `similarity_threshold` de um valor inicial até 0.99 (em incrementos de 0.01), permitindo observar como as métricas se comportam em diferentes níveis de exigência semântica. Isso possibilita identificar o threshold ótimo de operação — o ponto onde hit rate ainda é alto, mas a exigência de relevância semântica é suficientemente rigorosa para garantir qualidade.

7.2.5 Resultados do RAG



A análise conjunta dos gráficos indica que o **similarity_threshold = 0.77** representa o melhor ponto de equilíbrio entre qualidade de recuperação e precisão do sistema.

Nesse ponto:

- O **Hit Rate atinge 0.81**, indicando que a maioria das respostas relevantes ainda está sendo recuperada;
- O **MRR permanece em 0.66**, mostrando que os itens relevantes continuam aparecendo em posições relativamente altas no ranking;
- O **Precision@1 alcança 0.50**, sugerindo que metade das vezes o primeiro resultado retornado já é o correto.

Thresholds mais baixos aumentam levemente a cobertura, mas à custa de maior ruído. Por outro lado, valores mais altos tornam o critério excessivamente restritivo, provocando quedas abruptas em todas as métricas.

Assim, o valor **0.77** se mostra como um ponto de operação adequado para validar o pipeline de RAG, pois mantém uma taxa de acerto satisfatória sem comprometer de forma significativa a qualidade do ranqueamento ou a precisão do primeiro resultado.

8. Fluxo Completo de uma Interação

Ao receber uma pergunta do usuário, o seguinte fluxo é executado:

Passo	Componente	Ação	Saída
1	Memory_support	Busca top-4 memórias semânticas relevantes para a query	Bloco CONTEXT_MEMORY
2	ChromaDB (bge-m3)	MMR search: recupera top-20 chunks do COSO ERM 2017	Lista de 20 Document objects
3	RankContext (bge-reranker)	Cross-encoder re-rank dos 20 chunks	Top-6 chunks reordenados
4	build_prompt()	Monta prompt com template Llama 3.2 + memória + RAG + query	Prompt estruturado
5	Llama 3.2 3B (NF4)	Gera resposta com temperatura 0.3, max 512 tokens	Texto de resposta + sequences
6	CalculatePerplexity	Calcula log-probs e perplexidade sobre os tokens gerados	PPL + mean_logprob + pairs
7	Memory_support	Armazena par (query, response) na memória semântica persistente	Memória salva no ChromaDB

8	<code>_log_interaction()</code>	Persiste timestamp + system + query + response em JSONL	Log em llama_logs.jsonl
---	---------------------------------	---	-------------------------

9. Stack Tecnológico Completo

Categoria	Biblioteca / Ferramenta	Versão / Fonte	Uso
LLM Runtime	PyTorch	torch	Base de execução de todos os modelos neurais
LLM	Transformers (HuggingFace)	AutoModelForCausalLM	Carregamento e geração do Llama 3.2
Quantização	BitsAndBytes	BitsAndBytesConfig	Quantização NF4 4-bit do LLM
Embeddings	langchain-huggingface	HuggingFaceEmbeddings	Geração de embeddings para RAG e memória
Vector DB	ChromaDB	langchain-chroma	Armazenamento e busca vetorial (RAG + memória)
Re-Ranking	Transformers	AutoModelForSequenceClassification	Cross-encoder BAAI/bge-reranker-v2-m3
Document Loading	LangChain Community	TextLoader, PyPDFLoader	Carregamento do COSO ERM 2017
Text Splitting	LangChain Text Splitters	RecursiveCharacterTextSplitter	Fragmentação de documentos em chunks
Avaliação RAG	PyTorch + Transformers	RAGEvaluator (custom)	Hit Rate, MRR, Precision@1
Avaliação LLM	PyTorch	CalculatePerplexity (custom)	Perplexidade e log-probs por token
Logging	Python nativo	json, datetime	Registro de interações em JSONL

10. Considerações Técnicas e Decisões de Design

10.1 Uso de Temperatura Baixa (0.3)

A temperatura de 0.3 foi escolhida deliberadamente para reduzir a criatividade e aleatoriedade do modelo, favorecendo respostas mais determinísticas e fiéis ao contexto recuperado. Em domínios especializados como o COSO ERM, respostas consistentes e reproduzíveis são mais valiosas do que respostas criativas.

10.2 Isolamento Completo — Sistema On-Premises

Todo o sistema opera sem dependência de APIs externas. O modelo Llama é executado localmente com quantização NF4, os embeddings são gerados por modelos HuggingFace rodando em GPU local, e o ChromaDB é persistido em disco local. Isso é fundamental para ambientes corporativos com restrições de segurança e privacidade de dados.

10.3 Pipeline de Dois Estágios (MMR + Re-Rank)

A escolha de uma pipeline de recuperação em dois estágios (MMR para recall + cross-encoder para precision) representa um trade-off consciente entre custo e qualidade. A busca MMR recupera 20 chunks de forma eficiente, garantindo diversidade. O re-ranker, mais custoso computacionalmente (precisa processar 20 pares query-documento), é aplicado apenas sobre esse conjunto reduzido, tornando o custo total viável.

10.4 Gestão Automática de Memória

O sistema de memória implementa um ciclo automático de manutenção após cada interação: deduplicação semântica (evita redundância), purificação por limite temporal (mantém apenas as 300 mais recentes) e persistência em disco (ChromaDB). Esse ciclo garante que a memória permanece relevante e eficiente ao longo de longas sessões de uso.